

FanfareHub – Tutoriel d’architecture (MVC/DAO)

Projet TP9

16 mai 2026

1 Objectif

Ce document sert de fiche de compréhension du projet Java Web (Tomcat + JSP + Servlets + DAO) pour expliquer le code à l’oral.

2 Architecture globale

Vue (JSP)

Pages d’affichage et formulaires :
`connexion.jsp`, `inscription.jsp`, `menu.jsp`, `groupesPupitres.jsp`, `administration.jsp`, `evenements.jsp`, `evenementParticipation.jsp`.

Contrôleurs (Servlets, package `metier`)

- `AuthServlet` : connexion, inscription, logout
- `ChoixServlet` : groupes/pupitres
- `AdminServlet` : CRUD fanfarons + pupitres + groupes + événements (accès admin)
- `EvenementServlet` : événements + participation
- `ProfilServlet` : point d’entrée profil (minimal)

Filtre de sécurité (package `metier`)

- `AuthFilter (@WebFilter("/*"))` : protège toutes les routes.
- Bloque l’accès direct aux `.jsp` et force le passage par les servlets.
- Si non authentifié : autorise seulement `/Auth` et les ressources publiques (`css/js/images`).
- Si authentifié : empêche le retour sur login/signup (sauf `action=logout`).

Modèle (package `dao` + POJO package `metier`)

- DAO : `DbConnectionManager`, `FanfaronDAO`, `PupitreDAO`, `GroupeDAO`, `EvenementDAO`
- POJO : `Fanfaron`, `Pupitre`, `Groupe`, `Evenement`, `InscriptionEvenement`

3 Flux de requêtes importants

Connexion

`Auth?action=showLogin` → `connexion.jsp` → `AuthServlet.doPost(action=connexion)` → `FanfaronDAO.verifIdentif(...)` → session (login, role) → `sendRedirect("Profil")`.

Groupes/Pupitres (Q2)

`menu.jsp` → `ChoixServlet.doGet` (chargement options + choix actuels) → `groupesPupitres.jsp`.
Submit formulaire → `ChoixServlet.doPost` → `FanfaronDAO.saveChoix(...)` (transaction).

Administration fanfarons

Lien visible seulement si `role=admin`.

`AdminServlet` vérifie côté serveur (403 sinon), puis exécute les CRUD : `FanfaronDAO`, `PupitreDAO`, `GroupeDAO`, `EvenementDAO`.

Événements (Q4/Q5)

`EvenementServlet.doGet(action=list)` charge la liste.

CRUD événement autorisé seulement à la commission prestation via `EvenementDAO.isCommissionPrestationMem`

Participation fanfaron : `action=saveParticipation` → `EvenementDAO.replaceParticipation(...)`.

4 Mapping TP9 vers code

- Q2 (groupes/pupitres) : `ChoixServlet` + `groupesPupitres.jsp` + `FanfaronDAO.saveChoix`
- Q4 (CRUD événements) : `EvenementServlet` + `EvenementDAO.create/update/delete` + `evenements.jsp`
- Q5 (inscription événement) : `evenementParticipation.jsp` + `EvenementServlet.saveParticipation` + `EvenementDAO.findInscriptionsByEvenement`

5 POJO et DAO : comment ça marche

POJO

Un POJO est un objet Java simple (attributs + getters/setters/constructeur), sans logique SQL.
Exemple : `Fanfaron`, `Evenement`.

DAO

Un DAO contient les requêtes SQL. Il lit/écrit la base et transforme les lignes SQL en POJO.
Exemple : `FanfaronDAO.findByNomFanfaron()` retourne un objet `Fanfaron` construit depuis `ResultSet`.

Chaîne complète

Servlet (contrôleur) reçoit la requête → appelle DAO → DAO retourne POJO/listes POJO → Servlet met ça dans `request.setAttribute(...)` → JSP affiche.

6 Points d'attention pour l'oral

- Toujours distinguer **qui fait quoi** : JSP affiche, Servlet orchestre, DAO accède SQL.
- Sécurité : cacher un lien ne suffit pas ; la vérification est faite côté servlet **et** dans `AuthFilter`.
- Transactions : utiles pour garder une base cohérente (ex : `saveChoix`).
- Encodage UTF-8 : important pour les accents saisis dans les formulaires.

7 Résumé des classes et leur but

- `AuthFilter` : garde-barrière global des accès HTTP, redirections selon état de session.
- `AuthServlet` : gère login/signup/logout, crée et invalide la session.
- `ChoixServlet` : affiche et enregistre les choix groupes/pupitres de l'utilisateur connecté.
- `AdminServlet` : panneau admin central pour CRUD fanfarons/pupitres/groupes/événements.
- `Pupitre` : POJO simple (id, nom) pour transporter les données pupitre vers JSP/DAO.
- `DbConnectionManager` : singleton qui centralise l'accès JDBC à la base.
- `FanfaronDAO` : accès SQL fanfaron (auth, profil, choix, mapping id/login).

- `PupitreDAO` : accès SQL CRUD/listage des pupitres.
- `GroupeDAO` : accès SQL CRUD/listage des groupes.
- `EvenementDAO` : accès SQL CRUD événements + inscriptions + contrôles de commission.

8 Fiche des commandes techniques Servlet

- `res.sendRedirect("URL")` : redirection HTTP 302 vers une autre route (nouvelle requête client).
- `req.getRequestDispatcher("vue.jsp").forward(req, res)` : transfert interne serveur vers JSP (même requête).
- `res.sendError(code, "message")` : renvoie une erreur HTTP (ex : 400, 403, 404, 500).
- `req.setAttribute("cle", valeur)` : passe des données à la vue pour le **forward**.
- `req.getParameter("nom")` : lit un champ simple du formulaire/query.
- `req.getParameterValues("nom")` : lit plusieurs valeurs (checkbox multiples).
- `req.setCharacterEncoding("UTF-8")` : force l'encodage des paramètres entrants.
- `req.getSession(false)` : récupère la session sans en créer.
- `req.getSession(true)` : récupère ou crée une session.
- `session.setAttribute("cle", valeur)` : stocke un contexte utilisateur (login, rôle, etc.).
- `session.getAttribute("cle")` : lit une valeur de session.
- `session.invalidate()` : supprime la session (logout complet).
- `chain.doFilter(req, res)` : laisse passer vers le prochain filtre/servlet.

9 Methode pour expliquer une ligne aleatoire

Quand le prof pointe une ligne, utilise ce schema en 4 etapes :

1. **Contexte** : dire dans quel fichier on est (JSP, Servlet, DAO, POJO, Filter).
2. **Role local** : dire ce que fait exactement la ligne (lecture parametre, controle acces, SQL, affichage...).
3. **Impact global** : dire la consequence dans le flux utilisateur (redirige, bloque, enregistre, affiche).
4. **Risque evite** : dire pourquoi c'est utile (securite, null check, coherence BDD, UX).

Exemples d'explication rapide (1 phrase)

- `req.getSession(false)` : "on lit la session sans en creer, pour eviter des sessions fantomes".
- `res.sendRedirect("Profil")` : "on force un nouveau GET sur le controleur cible apres succes".
- `req.getRequestDispatcher(...).forward(...)` : "on rend la JSP dans la meme requete pour garder les attributs request".
- `if (!authenticated) ...` : "controle d'accès serveur, pas seulement cote interface".
- `dao.saveChoix(...)` : "la servlet delegue la persistance au DAO pour separer metier et SQL".
- `selectedPupitres.contains(...)` : "pre-cochage de l'UI a partir des choix deja en base".

10 Lecture guidee des fichiers cles

1) `AuthFilter.java`

- Calcule `path` a partir de `URI` - `contextPath` pour des regles robustes.
- Autorise assets publics + route `/Auth` sans session.
- Bloque acces direct aux `.jsp` (oblige le passage par les servlets).
- Redirige vers login si non connecte, vers profil si deja connecte sur login/signup.

2) AuthServicelet.java

- doGet : affiche login/signup et gere logout.
- doPost(action=connexion) : verifie identifiants via DAO, puis pose login/role en session.
- doPost(action=inscription) : valide formulaires (email/mdp confirmes, unicite) puis cree fanfaron.
- Sur succes, redirige vers Profil; sur echec, forward vers JSP avec message.

3) ChoixServlet.java + groupesPupitres.jsp

- doGet : charge listes pupitres/groupes + selections actuelles.
- doPost : enregistre les IDs coches puis recharge la page.
- JSP : checkboxes name="pupitres" / name="groupes" envoient des tableaux.
- Les libelles sont affiches a l'utilisateur, les IDs sont stockes techniquement.

4) EvenementServlet.java + evenements.jsp

- Liste des evenements accessible aux connectes.
- canManage active les boutons CRUD selon autorisation metier.
- Participation par evenement : lien vers page detail + inscription.

5) AdminServlet.java

- Point unique d'administration, protege par role=admin.
- CRUD fanfarons, pupitres, groupes, evenements.
- Charge toutes les listes a la fin pour re-render administration.jsp.

6) DAO et POJO

- DAO = SQL + mapping ResultSet.
- POJO = objets metier simples transportes entre DAO, Servlet et JSP.
- DbConnectionManager centralise la connexion JDBC (singleton).

11 Questions probables du prof (et reponses courtes)

- **Pourquoi un Filter + verif dans les servlets ?** Defense en profondeur.
- **Pourquoi redirect parfois et forward parfois ?** Redirect pour changer de route/eviter resoumission, forward pour afficher une vue avec attributs request.
- **Pourquoi DAO ?** Separation des responsabilites, testabilite, maintenance SQL.
- **Pourquoi stocker role/login en session ?** Etat d'authentification persistant entre requetes HTTP.
- **Pourquoi IDs en base mais noms dans la vue ?** Integrite technique en BDD + lisibilite utilisateur en UI.
- **Pourquoi setCharacterEncoding("UTF-8") ?** Preserver accents/symboles des formulaires.
- **Risque si acces direct JSP autorise ?** Contournement des controles de securite/metier.
- **Pourquoi null-check partout ?** Eviter exceptions et reponses 500 inutiles.

12 Fiche reponse en 20 secondes (lignes piegeuses)

1. HttpSession session = req.getSession(false); : on recupere la session seulement si elle existe, pour eviter d'en creer une inutilement.
2. session != null && session.getAttribute("login") != null : utilisateur considere connecte seulement si session + login present.

3. `if (path.endsWith(".jsp")) { ... }` : acces direct JSP interdit pour forcer le passage par les servlets (contrôle sécurité/métier).
4. `res.sendRedirect("Auth?action=showLogin");` : redirection HTTP, donc nouvelle requête navigateur vers login.
5. `req.getRequestDispatcher(...).forward(req, res);` : transfert interne serveur, même requête, attributs `request` conservés.
6. `req.setCharacterEncoding("UTF-8");` : preserve accents/caractères spéciaux envoyés par les formulaires.
7. `req.getParameterValues("pupitres");` : récupère toutes les valeurs cochées (tableau) pour des checkboxes multiples.
8. `selectedPupitres.contains(p.getId()) ? "checked" : ""` : pré-cochage de l'UI selon les choix déjà en base.
9. `daoFanfaron.verifIdentif(...);` : la servlet délègue la vérification des identifiants au DAO (séparation web/SQL).
10. `session.setAttribute("role", f.getRole());` : stocke le rôle en session pour appliquer les droits sur les requêtes suivantes.
11. `if (!isAdmin(...)) res.sendError(403,...);` : protection serveur des pages admin, indépendante de l'affichage des liens.
12. `DbConnectionManager.getInstance();` : singleton pour centraliser la configuration JDBC.
13. `DriverManager.getConnection(URL, USER, PASSWORD);` : ouvre une connexion PostgreSQL avec les paramètres configurés.
14. `<input type="hidden" name="action" ...>` : indique au servlet quel traitement exécuter sur le même endpoint.
15. `if (email == null || !email.equals(emailConfirm))` : validation métier pour bloquer une inscription incohérente.